Secure Password Policy Testing on a Sample Application

1.Introduction

Passwords remain one of the most widely used authentication mechanisms in modern web applications. Weak password policies, improper storage techniques, and insufficient authentication protections can expose systems to unauthorized access through brute-force attacks and credential compromise. Evaluating password security mechanisms is therefore essential to ensure application resilience against cyber threats.
This project focuses on testing password policies, authentication protections, and password storage mechanisms using a sample vulnerable application. The objective is to identify weaknesses and recommend improvements to strengthen security.

2. Objective

1)The main objectives of this project are:
2)To evaluate password complexity enforcement mechanisms.
3)To test authentication security against brute-force attacks.
4)To analyze password storage algorithms for weaknesses.
5)To assess hash strength using password cracking tools.
6)To recommend improvements for secure password handling.

3. Tools Used

The following tools were used in this project:
Burp Suite Community Edition -- Intercepting and analyzing authentication requests.
Hydra ---Automated brute-force password testing.
Hashcat --- Password hash strength testing and cracking.
OWASP Juice Shop --Sample web application for security testing.
Kali Linux -- Testing environment

4. Methodology

The project methodology consisted of multiple phases:
1. Setting up a sample web application with authentication functionality.
2. Testing password complexity requirements during account creation.
3. Intercepting login requests using Burp Suite.
4. Performing brute-force testing using Hydra and Burp Intruder.
5. Analyzing password storage algorithms.
6. Evaluating hash strength using Hashcat.
7. Identifying vulnerabilities and recommending mitigation strategies.

5. System Setup

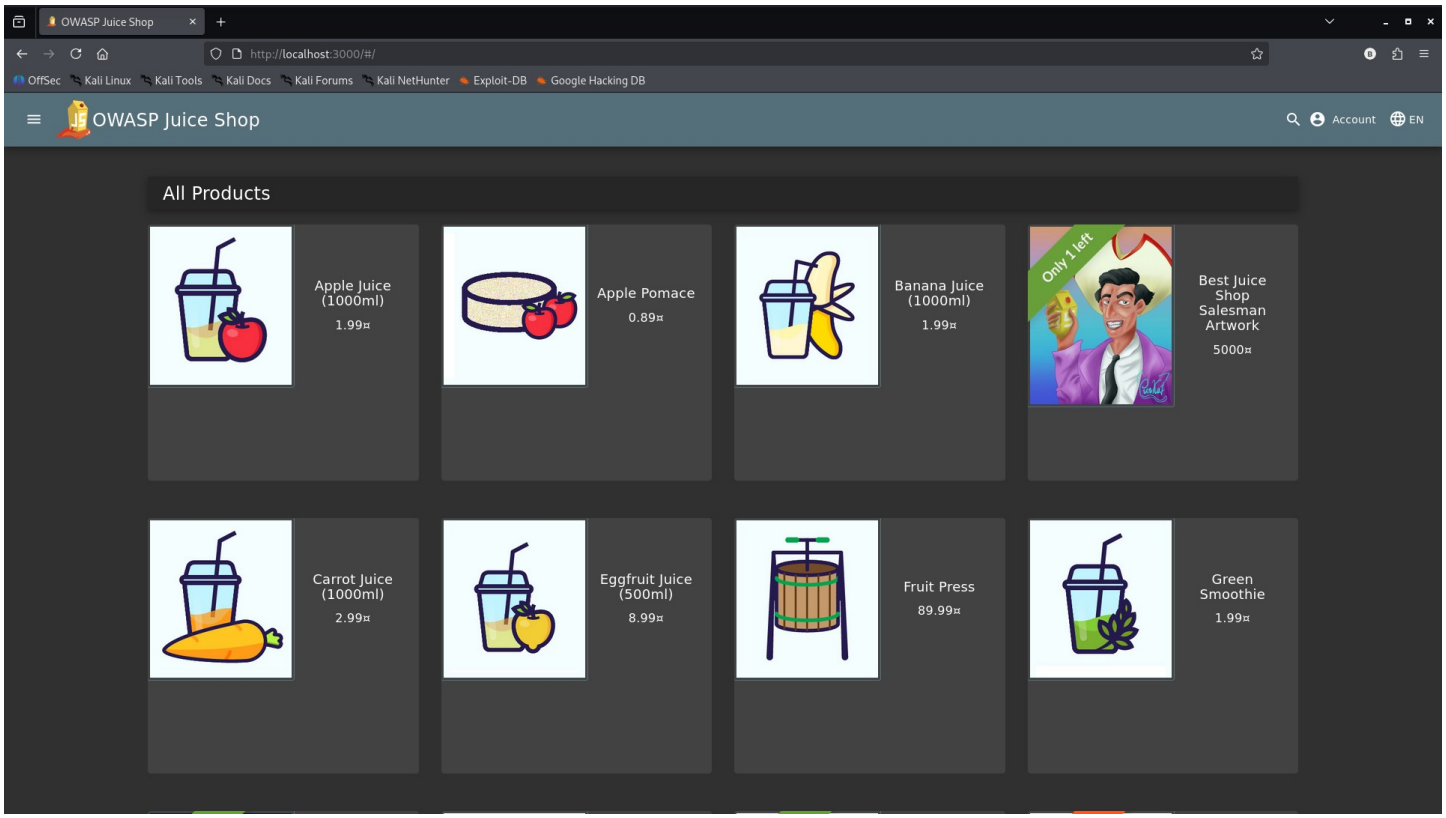
The OWASP Juice Shop application was deployed locally using Docker.

Command used:

```
┌──(bharath㉿kali)-[~]
└─$ sudo docker run -d -p 3000:3000 bkimminich/juice-shop
b3586a1eb5b2ecf847ab52ae7d7fc8ea63f9d5b9e434fcb060cf85c9d50035bc
```

The application was accessed via:

`http://localhost:3000`



6. Password Complexity Testing

Multiple password combinations were tested during account creation to evaluate enforcement mechanisms.

Test cases included:

Short passwords

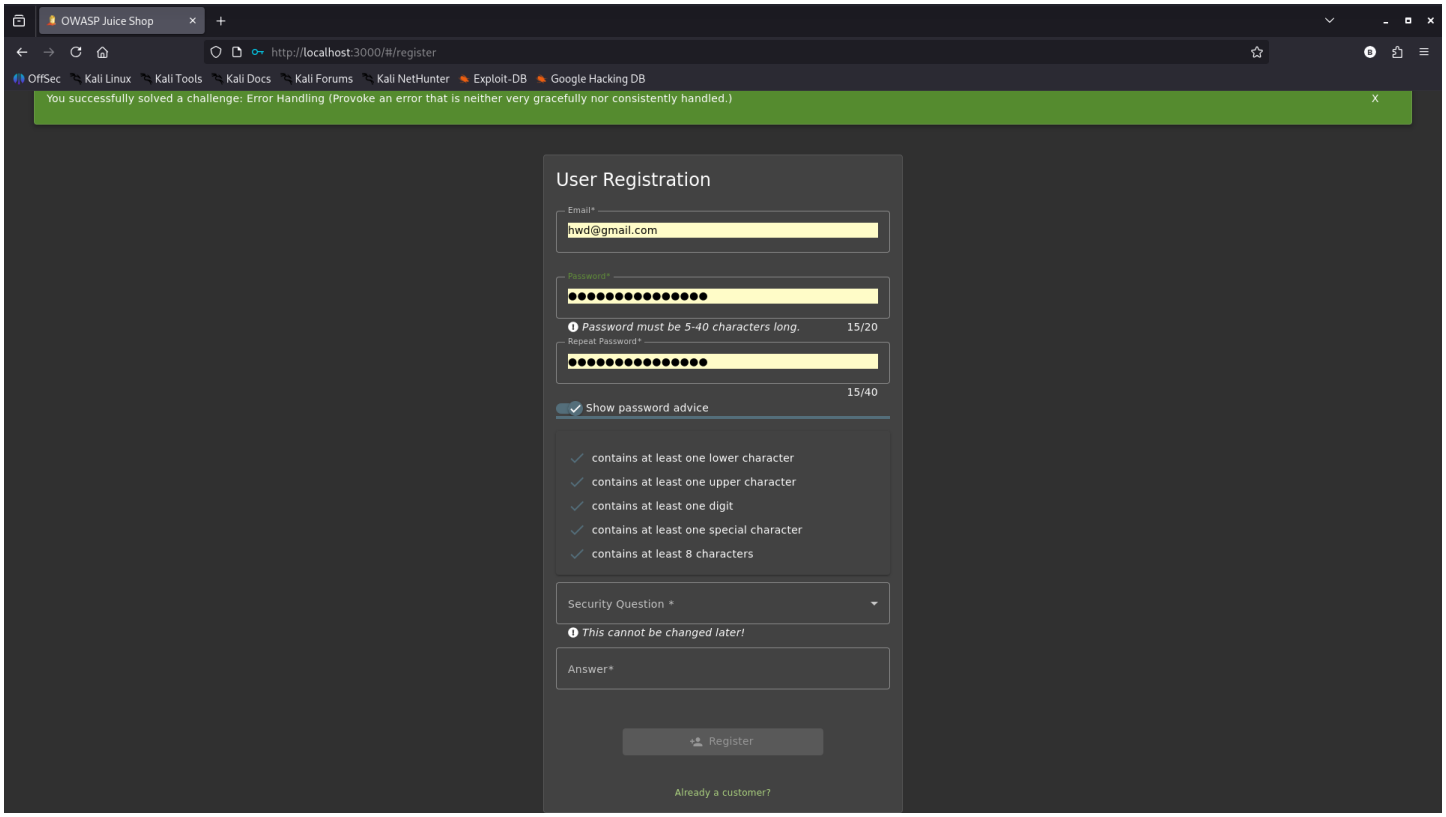
Numeric-only passwords

Alphabet-only passwords

Passwords without special characters

Strong passwords with mixed characters

Observations indicated that moderate password complexity requirements were enforced.



“For the above case, the password used is VkS8Vw4if#-^8QN.”

7. Authentication Testing Using Burp Suite

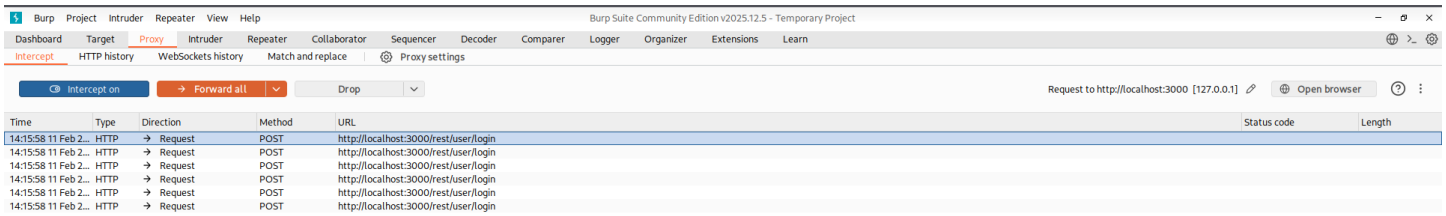
Burp Suite was configured as a proxy to intercept login requests between the browser and application.
The login request was captured successfully, revealing the authentication endpoint:

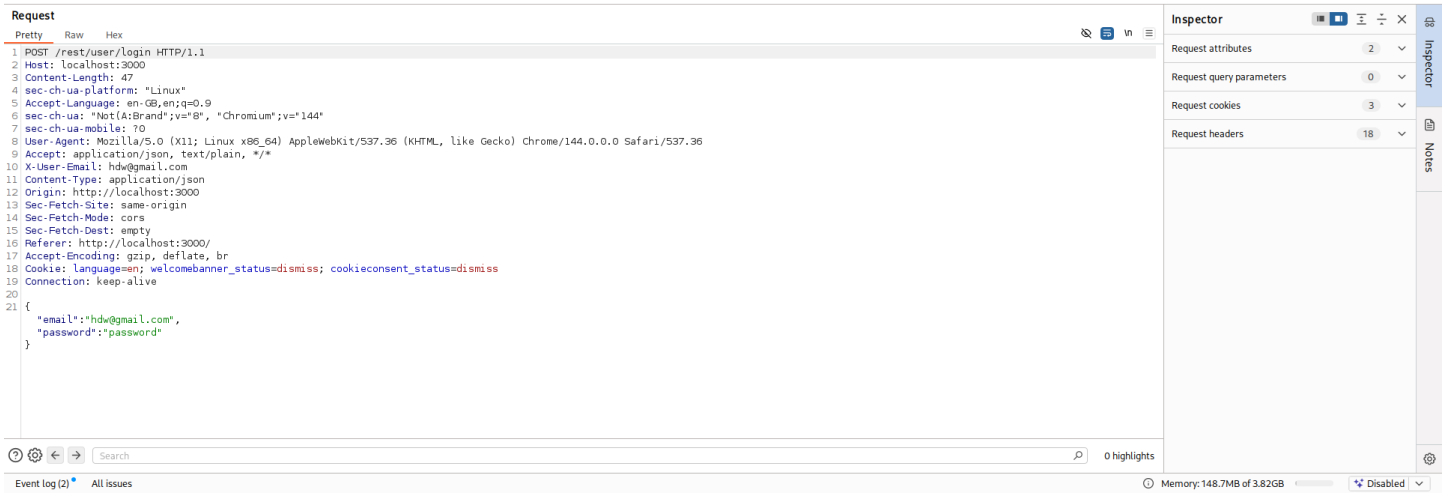
POST /rest/user/login

The request contained JSON parameters:

`Email,password`

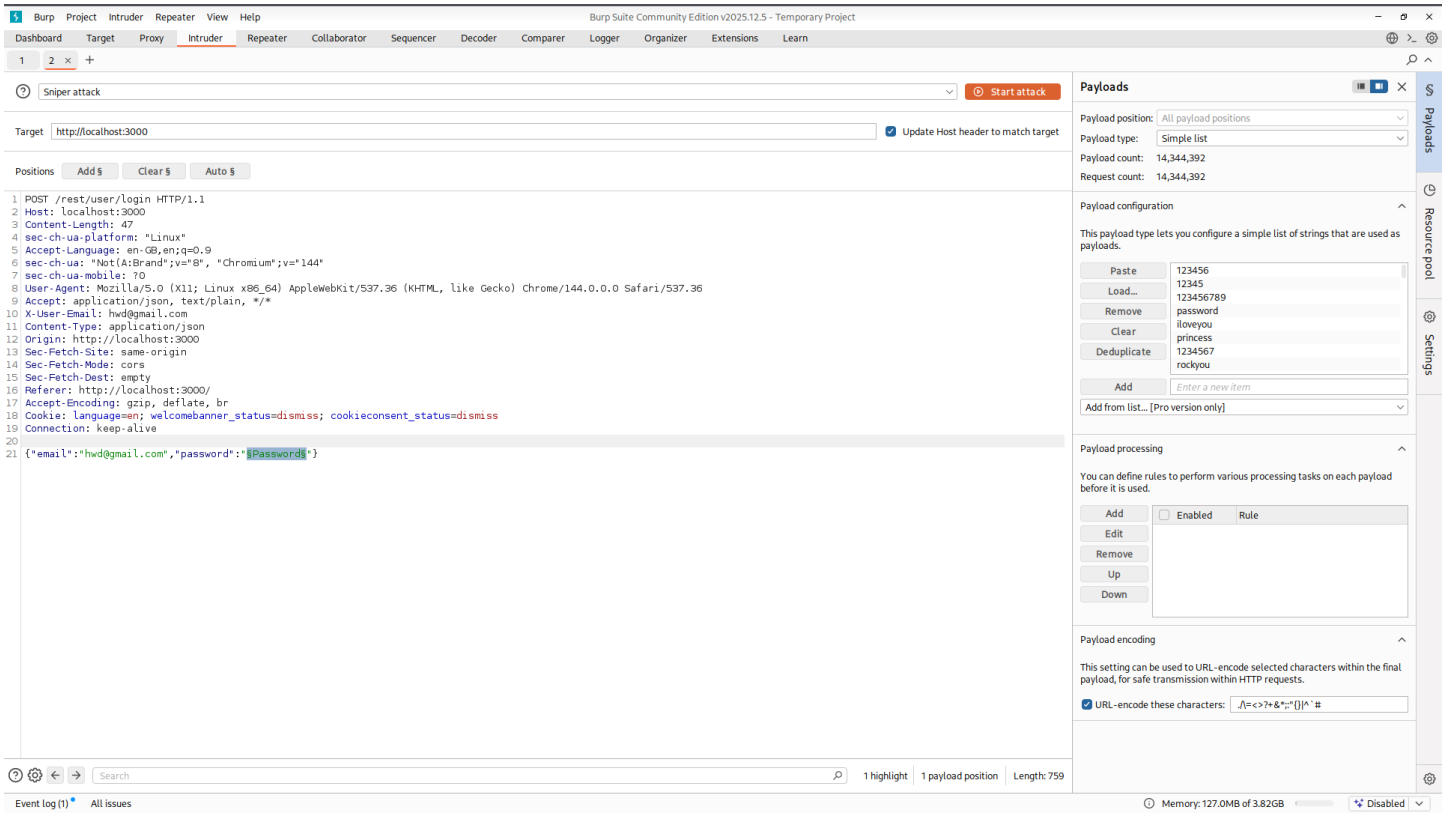
Intercept was enabled, and a login request with random credentials was captured. and cappture the session And sent it to intruder for brute force attack





Now the request is in the intruder tab select sniper attack set Payload Type as simple list and load rockyou wordlist.

Sniper attack is used to test multiple password values for a single parameter.After clicking start attack it will take some time for the brute force in burp is rather slow so the best recomendation is to Cmd line tools like hydra for speed and flexiblity



Burp Intruder was then used to automate multiple password attempts against the login endpoint.

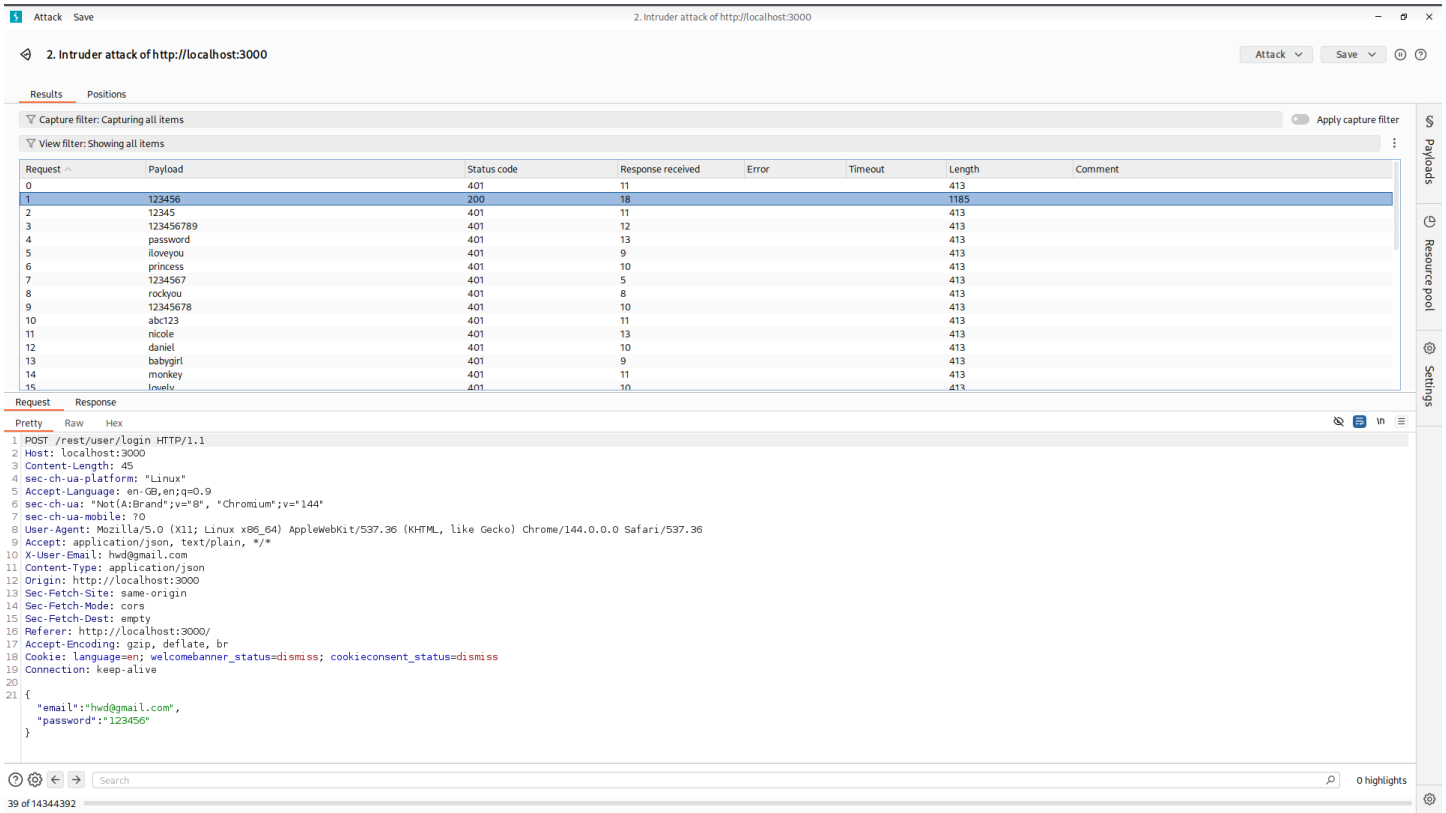
Observations showed:

Unlimited login attempts allowed

No account lockout mechanism

No CAPTCHA protection

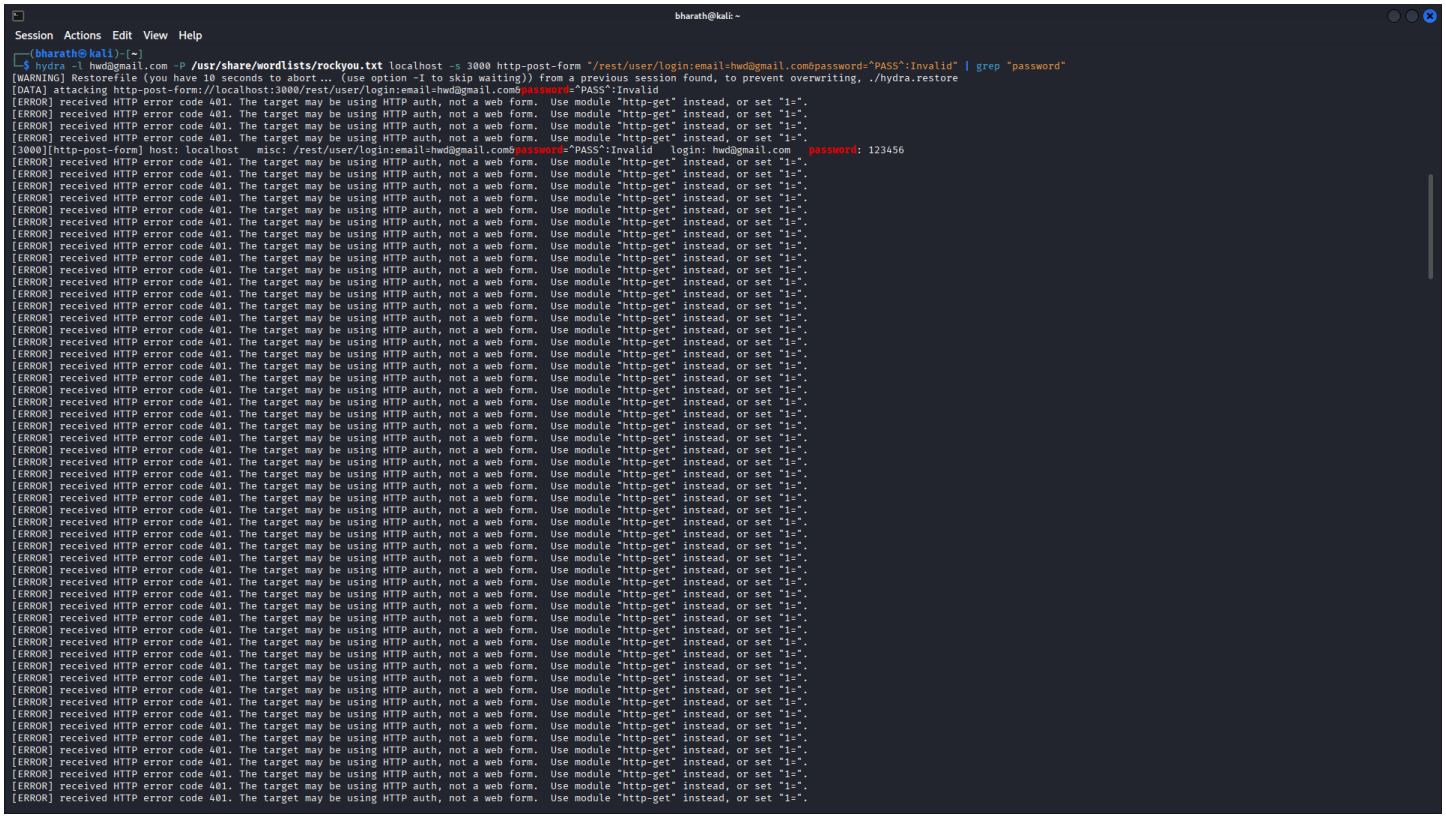
No noticeable delay between attempts



This indicates weak authentication protection.

8. Brute-Force Testing Using Hydra

Hydra was used to perform automated brute-force testing using a password wordlist.The application allowed continuous authentication attempts without blocking or restrictions.



This confirms the absence of brute-force protection mechanisms.

Hydra is a parallelized password cracking tool used to perform automated brute-force attacks against authentication systems.

9. Password Hashing Analysis

Source code analysis showed that the application uses the bcrypt hashing algorithm for password storage.

bcrypt is considered secure because:

It uses salting

It is computationally expensive

It resists rainbow table attacks

Hash that use below is :

$2b$10$KbQiP0n7HkQK1vZ1tcHTEO3e8DqRL3OjaxAg/P6MqxsVXni4eWhvC

```
bharath@kali: ~
Session  Actions  Edit  View  Help

┌──(bharath㉿kali)-[~]
└─$ nano hash.txt

┌──(bharath㉿kali)-[~]
└─$ hashcat -m 3200 hash.txt /usr/share/wordlists/rockyou.txt
hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian  Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]
====================================================================================================================================
* Device #01: cpu-haswell-13th Gen Intel(R) Core(TM) i9-13900H, 6788/13576 MB (2048 MB allocatable), 20MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 72
Minimum salt length supported by kernel: 0
Maximum salt length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Single-Hash
* Single-Salt

Watchdog: Temperature abort trigger set to 90c

Host memory allocated for this attack: 512 MB (6629 MB free)

Dictionary cache hit:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344385
* Bytes.....: 139921507
* Keyspace..: 14344385

Cracking performance lower than expected?

* Append -w 3 to the commandline.
  This can cause your screen to lag.

* Append -S to the commandline.
  This has a drastic speed impact but can be better for specific attacks.
  Typical scenarios are a small wordlist but a large ruleset.

* Update your backend API runtime / driver the right way:
  https://hashcat.net/faq/wrongdriver

* Create more work items to make use of your parallelization power:
  https://hashcat.net/faq/morework

[s]tatus [p]ause [b]ypass [c]heckpoint [f]inish [q]uit => s

Session..........: hashcat
Status...........: Running
Hash.Mode........: 3200 (bcrypt $2*$, Blowfish (Unix))
Hash.Target......: $2b$10$KbQiP0n7HkQK1vZ1tcHTEO3e8DqRL3OjaxAg/P6MqxsV...4eWhvC
Time.Started.....: Mon Feb 23 18:16:25 2026 (47 secs)
Time.Estimated...: Tue Feb 24 14:35:47 2026 (20 hours, 18 mins)
Kernel.Feature...: Pure Kernel (password length 0-72 bytes)
Guess.Base.......: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.#01........:      196 H/s (82.77ms) @ Accel:20 Loops:32 Thr:1 Vec:1
Recovered........: 0/1 (0.00%) Digests (total), 0/1 (0.00%) Digests (new)
Progress.........: 9200/14344385 (0.06%)
Rejected.........: 0/9200 (0.00%)
Restore.Point....: 9200/14344385 (0.06%)
Restore.Sub.#01..: Salt:0 Amplifier:0-1 Iteration:96-128
Candidate.Engine.: Device Generator
Candidates.#01...: ximenita -> kagandahan
Hardware.Mon.#01.: Temp: 62c Util: 81%

[s]tatus [p]ause [b]ypass [c]heckpoint [f]inish [q]uit => c

Checkpoint enabled. Will quit at next restore-point update.

Session..........: hashcat
Status...........: Aborted (Checkpoint)
Hash.Mode........: 3200 (bcrypt $2*$, Blowfish (Unix))
Hash.Target......: $2b$10$KbQiP0n7HkQK1vZ1tcHTEO3e8DqRL3OjaxAg/P6MqxsV...4eWhvC
Time.Started.....: Mon Feb 23 18:16:25 2026 (1 min, 13 secs)
Time.Estimated...: Tue Feb 24 16:20:21 2026 (22 hours, 2 mins)
Kernel.Feature...: Pure Kernel (password length 0-72 bytes)
Guess.Base.......: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.#01........:      181 H/s (80.37ms) @ Accel:20 Loops:32 Thr:1 Vec:1
Recovered........: 0/1 (0.00%) Digests (total), 0/1 (0.00%) Digests (new)
Progress.........: 13200/14344385 (0.09%)
Rejected.........: 0/13200 (0.00%)
Restore.Point....: 13200/14344385 (0.09%)
Restore.Sub.#01..: Salt:0 Amplifier:0-1 Iteration:992-1024
Candidate.Engine.: Device Generator
Candidates.#01...: john cena -> angel92
Hardware.Mon.#01.: Temp: 62c Util: 76%

Started: Mon Feb 23 18:16:02 2026
Stopped: Mon Feb 23 18:17:40 2026
```

bcrypt takes a long time to crack because it is intentionally designed to be slow and computationally intensive, which makes brute-force and password-guessing attacks significantly more difficult.

MD5 (Weak Hash)

An MD5 hash was generated and tested:

```
hashcat -m 0 md5.txt rockyou.txt
```

```
bharath@kali: ~
Session  Actions  Edit  View  Help

┌──(bharath㉿kali)-[~]
└─$ echo -n "password123" | md5sum
482c811da5d5b4bc6d497ffa98491e38  -

┌──(bharath㉿kali)-[~]
└─$ nano md5.txt

┌──(bharath㉿kali)-[~]
└─$ hashcat -m 0 md5.txt /usr/share/wordlists/rockyou.txt
hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian  Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]
====================================================================================================================================
* Device #01: cpu-haswell-13th Gen Intel(R) Core(TM) i9-13900H, 6788/13576 MB (2048 MB allocatable), 20MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Hash
* Single-Salt
* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

Host memory allocated for this attack: 517 MB (6665 MB free)

Dictionary cache hit:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344385
* Bytes.....: 139921507
* Keyspace..: 14344385

482c811da5d5b4bc6d497ffa98491e38:password123

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 0 (MD5)
Hash.Target......: 482c811da5d5b4bc6d497ffa98491e38
Time.Started.....: Mon Feb 23 18:20:32 2026 (0 secs)
Time.Estimated...: Mon Feb 23 18:20:32 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.......: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.#01........:   452.2 kH/s (0.32ms) @ Accel:1024 Loops:1 Thr:1 Vec:8
Recovered........: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.........: 20480/14344385 (0.14%)
Rejected.........: 0/20480 (0.00%)
Restore.Point....: 0/14344385 (0.00%)
```

The password was cracked instantly, proving that MD5 is weak and unsuitable for secure password storage.

10.Vulnerabilities Identified

The following vulnerabilities were identified:

Lack of brute-force protection

Missing account lockout mechanism

Absence of CAPTCHA after failed attempts

Unlimited login attempts allowed

Potential weak password acceptance

However, password storage using bcrypt was implemented securely.

11.Security Recommendations

The following measures are recommended:

Implement account lockout after multiple failed login attempts.

Add CAPTCHA protection after repeated failures.

Apply rate limiting to authentication endpoints.

Enforce strong password complexity requirements.

Enable multi-factor authentication (MFA).

Monitor suspicious login activity.

Continue using strong hashing algorithms like bcrypt or Argon2.

12.Conclusion

This project evaluated password security mechanisms in a sample web application by analyzing password complexity,authentication protections, and hash strength. Although the application uses secure bcrypt hashing for password storage, insufficient authentication protections expose it to brute-force attacks. Implementing stronger defensive mechanisms such as rate limiting, account lockout, and multi-factor authentication would significantly improve overall security.The comparison between MD5 and bcrypt demonstrates the importance of using modern hashing algorithms for password storage.

Risk Severity:

Vulnerability	Severity
No brute-force protection	High
No account lockout	High
Weak password acceptance	Medium